

Frontend Developer — Step 5: Build Projects

Expanded notes: requirements, MVP, architecture, UI/UX, APIs, CRUD, database, Git, testing, deployment and portfolio

1. Project Development & Lifecycle

Definition

Project development is the process of converting a real problem into a working, tested and deployable software product.

Lifecycle

Problem → Requirements → Planning → Design/Architecture → Implementation → Testing → Deployment → Feedback → Improvement.

A project is not complete just because it works locally. It should be understandable, tested and usable in its target environment.

2. Requirements, Functional/Non-functional & MVP

Functional requirement

Describes what the system must do. Example: a user can search careers.

Non-functional requirement

Describes quality or constraints such as performance, security, accessibility and responsiveness.

MVP definition

Minimum Viable Product is the smallest useful version that delivers the core value.

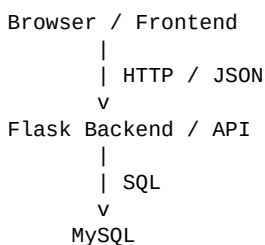
Example MVP:

1. Browse careers
2. Search careers
3. Open career details
4. View roadmap
5. Open resources

3. Planning & Architecture

Feature breakdown

Break large features into small tasks, identify dependencies, define data flow and API endpoints.



Separation of concerns

Frontend handles presentation and interaction; backend handles API/business logic/security; database stores structured data.

4. UI/UX, Responsive Design & Accessibility

Workflow

Wireframe → semantic structure → CSS layout → interactions → data/API → validation → responsive testing.

Responsive definition

An interface remains usable and readable across screen sizes using flexible layouts, relative sizing, Flexbox/Grid and media queries.

Accessibility

Use semantic HTML, labels, keyboard support, meaningful alt text, visible focus and readable contrast.

5. CRUD & HTTP APIs

CRUD definition

Create, Read, Update and Delete are the basic operations for many data-driven applications.

GET	/api/careers	# Read
POST	/api/careers	# Create
PUT	/api/careers/1	# Update
DELETE	/api/careers/1	# Delete

Request/response

Request contains method, URL, headers and sometimes body. Response contains status, headers and data.

Status groups

2xx = success, 4xx = client/request problems, 5xx = server-side failures.

6. Database Integration

Relational database

Tables store structured data in rows and columns. Primary key uniquely identifies a row. Foreign key represents a relationship.

```
careers
  id (PK)
  name

roadmap_steps
  id (PK)
  career_id (FK)
  title
```

Frontend should normally call the backend API; the backend controls database access. Database passwords never belong in frontend code.

7. Git, Testing & Debugging

Git workflow

```
git status
git add .
git commit -m "Add career details"
git push
```

Testing

Test normal and edge cases: empty data, invalid IDs, missing fields, network failures, duplicate submissions and mobile layouts.

Debugging

Reproduce → inspect → isolate → fix → retest original case → regression-test related features.

8. Deployment & Production

Definition

Deployment makes the application available in a production environment.

Production checklist

Correct API URLs, environment variables, CORS, database connection, static paths, build process, logs, HTTPS and error handling.

Secrets such as database passwords and private API keys must remain server-side and out of Git.

9. Portfolio Project Quality

Strong project

Real problem, useful features, clean responsive UI, meaningful backend/database integration when needed, error handling, Git history, README and live demo.

README

Problem, features, stack, architecture, setup, screenshots, API/database overview, live link and future improvements.

Explain your work

Be ready to explain architecture, database design, difficult bugs, trade-offs, security and what you personally implemented.

10. Project Ideas & Checklist

Ideas

Expense tracker, task manager, learning platform, quiz app, job tracker, weather dashboard, notes app, e-commerce prototype and career explorer.

Checklist

Requirements ✓ UI ✓ Responsive ✓ Git ✓ API ✓ Database ✓ Validation ✓ Errors ✓ Testing ✓ README ✓ Deployment ✓ Portfolio.